

评委一评分，签名及备注	队号： 20007	评委三评分，签名及备注
评委二评分，签名及备注	选题： A	评委四评分，签名及备注

题目：2048 游戏玩法与推广

摘要

本文就 2048 游戏展开，讨论了游戏 2048 的通关策略，介绍了建立通关模型的具体思想以及算法。基于所建立的通关模型分析了完成 2048 分数所需要的移动次数和成功概率，并通过 SPSS 和 Java 等软件验证了两个指标的有效性。利用模型信息先从 1*1 开始到 4*4 方格分析了所能达到的最大数值，针对分析出的数据，得出相应的递推关系，由一般到特殊整合出了 $N * N$ 方格所能达到的最大分数： $2^{(N*N)}$ 或 $2^{(N*N+1)}$ 。

针对问题一，建立了两个模型，分别通过 AI 算法（以一个 2048 源代码为例，分析其组成框架，为进一步改造 2048 这个游戏做好铺垫，并在 PC 平台完成了扩展）做出的模型和蒙特卡罗法整合出的模型算出了达到 2048 分数所需要的成功概率。模型一中，通过双博弈论模型，利用 AI 算法的思想中的 Mini max 算法和 Alpha - Beta 剪枝得到四个启发指标，针对启发指标用 Java 程序运行得到成功的概率和移动次数。根据调查 90 名游戏者得到一些统计数据，然后用 SPSS 软件进行处理和分析，得到实践的结果以验证理论结果的有效性。之后用蒙特卡罗方法在成功概率的基础上分析出了完成游戏目标所需要的移动次数利用 SPSS 程序对两部分方法做出的模型对通过实际操作得到的移动次数进行数据处理，来验证模型算出的结果是可行，是有效的。

针对问题二，本文通过对问题一结果的分析以及了解找到 2048 玩法的套路，然后由简单到复杂，由复杂到通用（从 1*1 到 4*4，从 4*4 到 $N * N$ ）到达最高分数，由此找到了递推关系式，并且得到了一般表达式。在 4*4 的表格中，继续玩，那么能玩到最大的数为 $2^{17} = 131072$ 。由于玩到最后一个时出现数字的概率是不确定的，是 2 或者是 4，所以最大分数的结果也是不确定的。在最后出现 2 的情况下最大分数为 $2^{(N*N)}$ ，最后数字为 4 的情况下最大分数为 $2^{(N*N+1)}$ ，并且通过实际操作得到的结果和推算的结果相互吻合。

关键字：移动次数 成功概率 AI 算法 蒙特卡罗法 SPSS

目录

1.问题重述	2
2.模型基本假设.....	2
3.符号说明	2
4.问题分析	3
4.1 问题一的分析.....	3
4.2 问题二的分析.....	3
5 模型的建立与求解.....	4
5.1 问题一的模型建立与求解	4
5.1.1AI 算法	4
5.1.2 达到目标解的移动步数分析	7
5.1.3 蒙特卡罗法与 2048 游戏	10
5.2 问题二的建立与求解.....	12
5.2.1 最大解分析——最后出数为 2	12
5.2.2 最大解分析——最后出数为 4	14
6.模型的评价与推广.....	15
6.1 问题 1 模型的评价与推广	15
6.2 问题 2 模型的评价与推广	16
7.参考文献	16
8.附录	17

2048 游戏玩法与推广

1. 问题重述

2048 是一款新兴而且火爆的智力游戏，游戏过程中只需控制所有的方块向同一方向移动，有相同数字的方块合并成他们的和，且每次移动完成时在空白的格子会随机出现 2 或 4，经过反复的移动操作，最终得到一个 2048 的方块格子即为胜利，若最终格子全满，没有出现 2048 则游戏失败。

问题一：正是因为这个游戏规则简单，很受人们青睐，但是却没人能给出一个通用的模型使之很容易完成到 2048，现请给出一个通用模型使之能达到 2048，并采用移动次数和成功概率两个指标来检验模型的有效性。

问题二：只要成功达到 2048 的玩家都知道，当达到 2048 后，游戏还可以继续进行，但是最大会达到多大呢？这需要学过的知识来讨论证明，如果将方格数从 $4*4$ 扩大到 $N*N$ 个，能达到的最大值是多少？

2. 模型基本假设

假设一 在问题一第一个模型中 $\alpha - \beta$ 剪枝的价值评价具体值假设按照编号 1-6 进行评价。

假设二 在问题一第二个模型中采用的蒙特卡罗法将长度 1 假设为出现数字的位置，将出现数字的位置距离移动点的长度假设为方格数 a 。

假设三 在问题二中最大值的运行过程中假设采用“蛇字形”移动方法。

3. 符号说明

N	单个样本的个案数
n	移动数字达到目标的次数
Sig	双侧 T 检验的概率
df	自由度
a	移动到的位置
m	移动出现数与已有数合并次数

x	随机数
σ_i	移动次数方差
μ	移动次数的期望值
E	合并总数参数的期望值
γ_i	绝对容差
y	合并总数参数
C	容差等级
P	成功概率
Q	偏离目标值的平均损失
L	造成的次数损失

4. 问题分析

4.1 问题一的分析

处理该题的首要步骤就是对这个游戏 2048 的本质和游戏规则具有很好的了解。问题一拟建立两个模型，模型一，用 AI 算法中 *Mini max* 算法和 *Alpha - Beta* 剪枝两个方法进行讨论。*Mini max* 算法是一种悲观主义算法，决策时首先考虑的是选取每一策略时所可能出现的最坏结果，然后从这些最坏结果中选择最好的结果，然后做出相应的格局树。*Alpha - Beta* 剪枝分析得到了四个指标，AI 算法通过 Java 运行可以算出成功合成 2048 的概率超过 90% (91.739%)，模型二，在一定的前提下，利用蒙特卡罗法可得，移动数字决策取决于容差等级，记作 $a(a > 0)$ ，于是完成游戏所需步数为： $p = \frac{2l}{a\pi}$ 。对此，对 90 位玩家进行了调查，得到了 81 位玩家通关数值，用 SPSS 对得到的通关的 81 个玩家的数值进行了求解，最后得到模型移动次数的均值和概率。然后结合附件中资料，更好的验证了模型的有效性。

4.2 问题二的分析

得到 2048 之后, 游戏还可以继续玩, 为了得到最大的数值, 分别从游戏 2048 所涉及到的创建的源泉和游戏规则得分数的一定规律。通过源代码的分析, 最大解情况分析要从概率方面考虑, 分两种情况考虑。第一种情况是在排满方格前最后一个数字恰好是 2, 第二种情况是在排满方格前最后一个数字恰好是 4, 分别得到 $2^{(N * N)}$ 和 $2^{(N * N + 1)}$ 。

5 模型的建立与求解

5.1 问题一的模型建立与求解

5.1.1 AI 算法:

AI 算法对与 2048 这款游戏, 可以以较大概率甚至高于 90% 概率赢得 2048 这款游戏。2048 所需的 AI 算法将主要分为两个部分, 第一部分介绍其中用到的基础算法, 即 *Mini max* 和 *Alpha - Beta* 剪枝; 第二部分分析我们如何具体实现 AI 算法——2048 的运行。

模型算法分析——基础算法:

2048 游戏本质上可以抽象成“信息一致”双人对弈模型, 玩家用手指可以向任意四个方向中一个移动, 之后游戏屏幕在空格的任意一处产生数字 2 或 4。“信息一致”指的是在任何时刻对弈双方对格局的信息内容是完全一致的, 对弈策略仅依赖接下来对这个格局的推理。

Mini max 算法:

解决博弈论问题方法众多, 就自然想法来说一般就是将格局理想化成一棵树, 树上面有很多分叉和节点, 树上的每一个节点表示一种格局。*Mini max* 也是相通的, 它通过对之前父格局为根的格局树搜索来确定下一步对子格局的选择。而一切格局树搜索算法的最基本核心都是对每个格局价值的评价。

基本步骤如下:

(1) 首先确定最大搜索深度 H , 而且要知道 H 可能达到最终格局, 也可能是一个中格局。

(2) 在最大深度为 H 的格局树叶子节点上, 使用预定义的价值评价函数对叶子节点价值进行评价。

(3) 从下面到上面为非叶子节点赋值。其中 *max* 节点取子节点最大值, *min* 节点取子节点最小值。

(4) 每次轮到我方时 (此时必处在格局树的某个 *max* 节点), 选择价值等于此 *max* 节点价值的那个节点路径。

Alpha - Beta 剪枝:

从根节点开始, 使用 *Alpha - Beta* 剪枝方法的步骤:

(1) 首先将根节点的 *Alpha* 和 *Beta* 分别初始化为 $-\infty$, 和 $+\infty$;

(2) 优先搜索第二层第一个孩子 (不是叶子节点), 所以 *Alpha* 和 *Beta* 继承

自父亲节点，所以它们也分别为 $-\infty$ ，和 $+\infty$ ；

(3) 搜索第三层的第一个孩子，*Alpha* 和 *Beta* 同上分别为 $-\infty$ ，和 $+\infty$ ；

(4) 搜索第四层，到达叶子节点，采用评价函数得到此节点的评价值为 1，如图 1 所示。

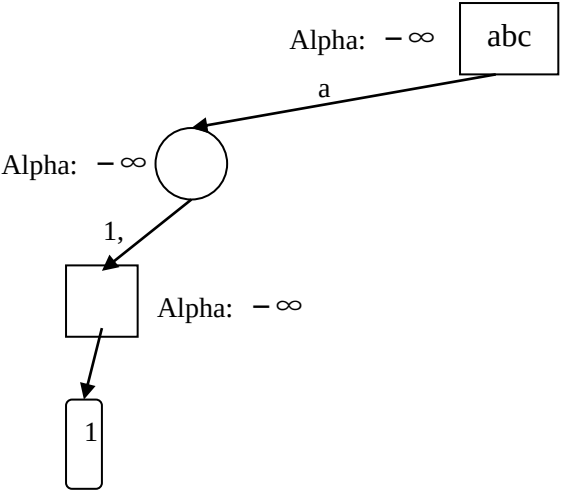


图 1 *Alpha* - *Beta* 剪枝 1

(5) 此叶节点的父节点为 *max* 节点，因此更新其 *Alpha* 值为 1，表示此节点取值的最小值为 1。

(6) 再看另外一个子节点，其值为 4，大于当前 *Alpha* 值，因此将 *Alpha* 值更新为 4。

(7) 此时第三层最左边节点所有子树搜索完毕，作为 *max* 节点，更新其真实值为当前 *Alpha* 值为 4。

(8) 由于其父节点（第二层最左节点）为 *min* 节点，因此更新其父节点 *Beta* 值为 4，表示这个节点取值最多为 4，如图 2 所示。

(9) 搜索第二层最左节点的第二个孩子及其子树，按上述逻辑，得到值为 5（注意第二层最左节点的 *Beta* 值要传递给孩子）。由于 5 大于 4，不更新 *min* 节点的 *Beta* 值，如图 3 所示。

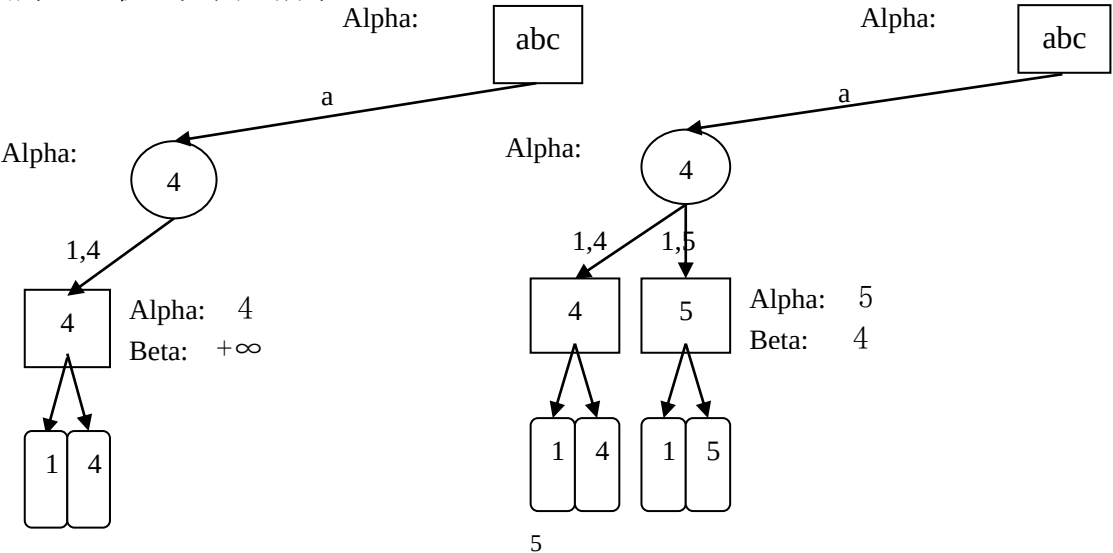


图 2 *Alpha* - *Beta* 剪枝 2

图 3 *Alpha* - *Beta* 剪枝 3

(10) 搜索第二层最左节点的第三个孩子。当看完第一个叶子节点后，发现第三个孩子的 $\text{Alpha} - \text{Beta}$ ，此时表示这个节点下不会再有更好解，于是剪枝。

后面搜索方法同上。

关于 Mini max 和 $\text{Alpha} - \text{Beta}$ 都是针对信息对称的轮流对弈问题，抽象游戏规则：

(1) 游戏玩家。每次可以选择上、下、左、右四个行棋策略中的一种（某些格局会少于四种，因为有些方向不可走）。行棋后方块按照既定逻辑移动及合并，格局转换完成。

(2) 计算机。在当前任意空格里放置一个方块，方块的数值可以是 2 或 4。放置新方块后，格局转换完成。

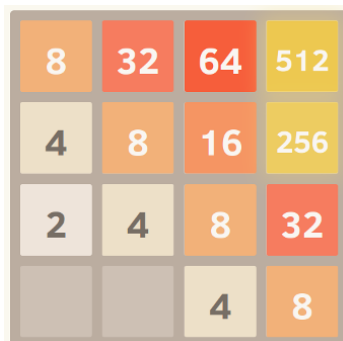
(3) 胜利条件：出现某个方块的数值为“2048”。

(4) 失败条件：格子全满，且无法向四个方向中任何一个方向移动（均不能触发合并）。

在 2048 中，当前格局的价值是一个算法的核心，“好”的格局容易引向胜利，而“坏”的格局引向失败，因此为了提升胜利的概率，需要考虑以下几个因素：

单调性：

单调性指方块各个方向的数字保持单调递增或者是递减，这样可以使大数和小数分开，从而使大数或小数依顺序在一起，形成好的格局，如图 4 所示。



8	32	64	512
4	8	16	256
2	4	8	32
		4	8

图 4 2048 单调性格局

平滑性：

平滑性指相邻格子的数字的差值，其差值应尽可能的小，从而得到较“好”的格局。

空格数：

当出现满格时，游戏失败，因此，空格越少对玩家越不利，为了出现“好”的格局，应该让空格尽量多。

孤立空格数：

孤立空格数指空格被分开的程度，要想得到“好”的格局，需要是空格尽量

的密集，空格越分散的则格局越差。

以上是关于 2048 – AI 在评价格局时，对这些启发指标采用加权策略运行的。具体代码如下见附录代码 1。

针对对方选择的剪枝这个程序，除了采用 *Alpha – Beta* 剪枝外，在 *mix* 节点还采用了另一种剪枝，即只考虑对方走出让格局最差的那一步（而实际 2048 中计算机的选择是随机的），而不是搜索全部对方可能的走法。这是因为对方所有的可能的选择为“空格数×2”，如果全部搜索的话会严重限制搜索深度。相关剪枝代码见附录代码 2。

在搜索深度方面，2048 – AI 并没有限制搜索的最大深度，而是限制每次“思考”的时间。这里设定了一个超时时间，默认为 100ms，在这个时间内，会从 1 开始，搜索到所能达到的深度。相关代码见附录代码 3。

通过 AI 算法通过 Java 运行可以算出成功合成 2048 的概率超过 90% (91.739%)。

5.1.2 达到目标解的移动步数分析

按照递推关系分析，得到的 2048 是靠一个个生成的 2 和 4 加起来的。由于随机生成 2 的次数与 4 相比，2 出现的次数远大于 4，因此在这里为了简化讨论，我们假设每一次移动产生的方格都是 2。这样的话，我们有如下的递推关系：

产生 2048 需要移动 1 次（2 个 1024 相加）

产生 2 个 1024 需要移动 2 次（4 个 512 分别相加）

产生 4 个 512 需要移动 4 次（8 个 256 分别相加）

产生 8 个 256 需要移动 8 次（16 个 128 分别相加）

产生 16 个 128 需要移动 16 次（32 个 64 分别相加）

产生 32 个 64 需要移动 32 次（64 个 32 分别相加）

产生 64 个 32 需要移动 64 次（128 个 16 分别相加）

产生 128 个 16 需要移动 128 次（256 个 8 分别相加）

产生 256 个 8 需要移动 256 次（512 个 4 分别相加）

产生 512 个 4 需要移动 512 次（1024 个 2 分别相加）

因此，总移动次数为： $512+256+128+64+32+16+8+4+2+1 = 1024 - 1 = 1023$ 次。

在游戏运行过程中很可能出现同时合并好几个解的情况。例如，在图 5 中，向上（或者向下）移动就能够同时合并 4 个数：

4	2	2	
4	2	2	
4			
4			

图 5 2048 特殊性例子

这样的合并显然会减少总的合并和移动次数。但是这种情况并不是每次都发生，同时还会经常发生移动但不合并任何数的情况。因此，这种并行合并的情况以及移动不合并的情况在这个模型中不进行讨论。

对于 2048 这个数值，移动次数为 $2048 / 2 - 1$ 次。

通过规律可以得到对于 2048 数值的理论最小移动次数为 1023 次。

所以，对于 2048 游戏 *Java* 运行成功概率为 91.739%，在成功的情况下通过 *Java* 运行可以得到移动次数均值为 1075 次，与 1023 次差距不大，因此有效。

下面根据调查 90 名游戏者得到以下统计数据，如表 1 所示

表 1 完成游戏共需要的移动次数和是否成功

次数	1026	1109	1083	1047	1038	1048		1025	1102	1033
是否成功	是	是	是	是	是	是	否	是	是	是
次数	1049	1033	1147	1232	1098	1132	1034	1065	1024	1101
是否成功	是	是	是	是	是	是	是	是	是	是
次数	1076		1201	1034	1045	1063		1108	1047	1024
是否成功	是	否	是	是		是	否	是	是	是
次数	1075	1124	1038	1038	1231		1026	1076	1029	1042
是否成功	是	是	是		是	否	是	是	是	是
次数	1027	1032	1067	1083	1051	1042	1025	1054	1034	
是否成功	是	是	是	是	是	是	是		是	否
次数	1054	1032	1092	1051	1086	1124	1101		1185	1102
是否成功	是	是		是	是	是	是	否	是	是
次数		1043	1089	1143	1102	1043	1024	1053	1056	1066
是否成功	否	是	是	是	是	是	是	是	是	是
次数	1103	1034	1204	1104	1064		1054	1046	1053	1043
是否成功	是	是	是	是	是	否		是		是
次数	1183	1203		1067	1104	1073	1035	1102	1221	1042
是否成功	是		否	是	是	是	是	是	是	是

利用表一中的数据，通过 SPSS 进行 *t* 检验，结果如下表 2、3：

表 2 单个样本统计量

单个样本统计量				
	<i>N</i>	均值	标准差	均值的标准误
VAR00001	81	1077.7654	53.33462	5.92607

表 3 单个样本检验

单个样本检验						
	检验值 = 1075					
	<i>T</i>	<i>df</i>	<i>Sig.</i> (双侧)	均值差值	差分的 95% 置信区间	
					下限	上限
VAR00001	467	80	642	2.76543	-9.0278	14.5587

结果分析：（检验方法不准确。0.465>0.05 的比较不能说明问题。）

如表格 2 所示，呈现了单个样本统计量的值，包括单个样本的个案数(*N*)、均值、标准差、均差的标准误。其中成功完成次数为 81 次，平均每次移动次数为 1078 次，标准差为 53.33462，均值的标准误为 5.92607。

此单样本 *T* 检验结果报表如表格 B 所示，包括 *t* 值、自由度(*df*)、检验的概率(*Sig*)、均值差值、差分的 95%置信区间。。其中 *t* 检验的 *t* 值为 0.467，自由度为 80，双侧 *t* 检验的概率 *Sig.* (双侧) 的值为 2.76543，置信区间为 [-9.0278, 14.5587]。假设玩家游戏成功每次所移动次数与 1075 无关。因显著性水平为 0.05，而 0.465>0.05, 因此假设不成立，即玩家每次成功时所移动的次数与 1075 无显著差异。

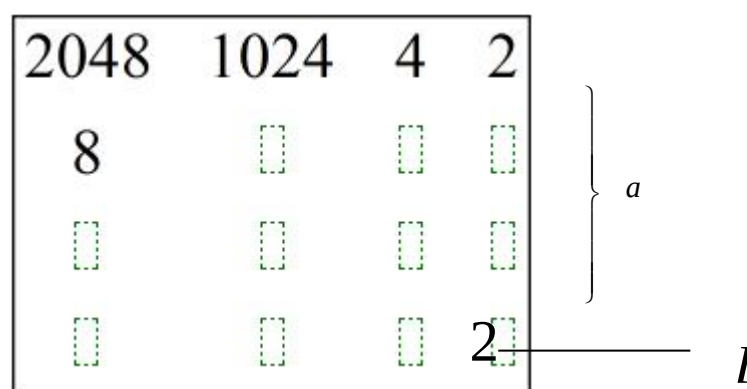
T 检验对应的置信区间公式 $\bar{X} - t_{\alpha/2} \sigma_x < \mu < \bar{X} + t_{\alpha/2} \sigma_x$ ，则玩家每次成功所以移动的次数 95%的置信区间=均值（1077.7654）±1.96×标准误（5.92607），即 [1066.15, 1089.38]，这个置信区间与给定的均值之差为 [-11.6154, 10.6146]，这个和上限和下限的数据相对应，但是有微误差。

通过表 1 得到的数据对 *Java*（双侧）代码运行的 2048-*AI* 算法得到移动次数和成功概率两项结果进行分析验证，说明了 *Java* 运行结果的有效性，所以数据是有效的。

5.1.3 蒙特卡罗法与 2048 游戏

在 2048 游戏中最主要的随机变量是移动后 2 或者 4 出现的位置，玩家每次移动后要根据位置与数字值来确定下一步走法。适合游戏进行下去到达 2048 数值的策略是制定指定一下数字出现位置的优势路线与劣势路线的具体概率。一般情况是优势路直接可以合并数字，劣势路要多走一步才能合并数字。在这种情况下成功概率 P 与在成功情况下的移动次数 n 构成到达 2048 的通用模型。

游戏桌面上有 16 个相同大小的方格，每次移动后随机出现的数的位置距离想要移动到的位置为 $a(a > 0)$ ，每次移动后出现数的位置用 1 表示，随机地像平行线的平面上掷去，求解出现的数与现有数合成的概率。由蒙特卡罗法可以算出概率为 $p = \frac{2l}{a\pi}$ ，而通过移动次数 n （ n 是移动数字达到目标的次数），观察到移动出现数与已有数合并次数 m 。取 m/n 为 p 的近似值，可以算出 π 的近似值。可以想象当移动次数越来越多时计算结果越来越准确。



采用蒙特卡罗方法主要来分析难以定量的 2048 游戏成功概率模型以及计算无法确定解析结果的 2048 一定次数的定性模型。

用这套方法解决 2048 游戏题，根据随机变量遵循的分布规律选出的 2 或 4 具体概率值，与抽样方法类似。在计算机上运行这款游戏产生的各种分布的随机数都会按照一定的确定性方法产生的伪随机数。

随机数的生成：

随机数就是随机变量的实现值，即样本。对于出现 2, 4 的随机结果可以用离散随机变量的概率函数来描述。如果我们需要随机变量的一个或多个实现值（样本），可以用实际移动然后记录其结果的方法来得到，然而这种方法具有相当的局限性。用数学程序来产生随机变量拟样本，即看上去是随机出现的，但并非真正的随机的实现值（样本）。由此产生递推公式，但这些拟随机数并没有明显的规律，当给以适当的伸缩后，它们将接近于 $[0, 1]$ 区间的均匀分布，在程序设计和软件包中通常用 RND 来表示 $[0, 1]$ 上的均匀分布的随机变量的样本，并由此可以构造产生另外的随机变量随机数（样本）。

举例说明，可以从式子 $X = a + (b - a)RND$ 给出区间 $[a, b]$ 上的连续均匀分布的随机变量。如果要生成带参数 λ 的指数分布的随机数，可以用

$X = -(1/\lambda)\ln(RND)$ 。要生成平均值为零，标准差为 1 的正态分布，可以用

$X_1 = [-2\ln(RND_1)]^{\frac{1}{2}} \cdot \cos(2\pi RND_2)$ 和 $X_2 = [-2\ln(RND_1)]^{\frac{1}{2}} \cdot \sin(2\pi RND_2)$ 给出 X 的随机数，令 $X = \sigma X_2 + \mu$ 或 $X = \sigma X_1 + \mu$ ，可以生成 $N(\mu, \sigma)$ 型的正态分布随机变量的随机数。

计算过程如下：

(1) 单独样本数参数 $x_1, x_2, \dots, x_n$ 为服从相互独立的随机变量，期望值与均方差分别记为 $\bar{x}_i$ 和 σ_i ，绝对容差记作 $\gamma_i = 3\sigma_i$ ，相对容差记作 $t_i = \frac{\gamma_i}{x_i}, (i=1, 2, \dots, n); t = (t_1, t_2, \dots, t_n)$ 。

(2) 合并总数参数 y 由 $x_1, x_2, \dots, x_n$ 决定，记作 $y = f(x_1, x_2, \dots, x_n)$ ，由于 x_i 偏离 $\bar{x}_i$ 很小，可以在 $\bar{X} = (\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$ 处对 f 作 *Taylor* 展开，并略去三次及三次以上各项有

$$y = f(\bar{X}) + \sum_{i=1}^n d_i (x_i - \bar{x}_i) + \frac{1}{2} \sum_{i=1}^n e_{ii} (x_i - \bar{x}_i)^2 + \sum_{i=1}^{n-1} \sum_{j=i+1}^n e_{ij} (x_i - \bar{x}_i)(x_j - \bar{x}_j)$$

其中 $d_i = \left. \frac{\partial f}{\partial x_i} \right|_{\bar{x}}, e_{ii} = \left. \frac{\partial^2 f}{\partial x_i^2} \right|_{\bar{x}}, e_{ij} = \left. \frac{\partial^2 f}{\partial x_i \partial x_j} \right|_{\bar{x}}$ ；我们仅用 x_i 的二阶矩即期望值和均

方差，而忽略 x_i 的三阶和三阶以上矩。于是随机变量 y 的期望值为

$$E_y = f(\bar{X}) + \frac{1}{2} \sum_{i=1}^n e_{ii} \sigma_i^2 \text{ 方差 } \sigma_y^2 = \sum_{i=1}^n e_{ii} \sigma_i^2 \text{ 或用 } t_i \text{ 表示为}$$

$$E_y = f(\bar{X}) + \frac{1}{18} \sum_{i=1}^n e_{ii} \bar{x}_i^2 t_i^2 \quad (1)$$

$$\sigma_y^2 = \frac{1}{9} \sum_{i=1}^n (d_i \bar{x}_i t_i)^2 \quad (2)$$

(3) 由于 y 偏离目标值 y_0 （数字出现的位置不是想要的最优位置）造成的次数损失（无用移动次数增加）记作 $L(y)$ ，由模型数据可设 $L(y)$ 与 $(y - y_0)^2$ 成比。即构造次数损失的近似值，连续的解析表达式 $L(y) = (y - y_0)^2$ ，这里 k 为待定的任意常数。

(4) 大量移动数字时，造成移动次数损失的价值平均可近似表达为：

$$Q(\bar{x}, t) = EL(y) = kE(y - y_0)^2 = k[(EY - y_0)^2 + \sigma_y^2]$$

即

$$Q(\bar{x}, t) = k \left\{ [f(\bar{X}) - y_0 + \frac{1}{18} \sum_{i=1}^7 e_{ii} \bar{x}^2 t_i^2]^2 + \frac{1}{9} \sum_{i=1}^7 (d_i \bar{x} t_i)^2 \right\} \quad (3)$$

(5) 移动数字决策取决于容差等级 t_i ，记作 $C_i(t_i)$ ，于是完成游戏所需步数为：

$$C(t) = \sum_{i=1}^n c_i(t_i) \quad (4)$$

(6) 完成 2048 游戏的成功概率用 P_h 表示， $P_h = P\{|y - y_0| \leq 0.1\}$ ；多余移动次数的概率用 P_c 表示， $P_c = P\{0.1 < |y - y_0| \leq 0.3\}$ ；无用移动次数的概率用 P_f 表示， $P_f = P\{|Y - Y_0| > 0.3\}$ ，由于 y 偏离目标值的平均损失为

$$Q(\bar{X}, t) = EL(y) = 0 \cdot P_h + 1000 P_c + 9000 P_f \quad (5)$$

(7) 综合考虑 y 偏离 y_0 造成的次数损失和出现数字，将本问题的目标函数定义为完成游戏时所需的总次数。（完成游戏必须在成功的前提下）

(8) 我们把数学规划和数值模拟拟合，采用逐步求精的迭代求解方法。先确定参数的标定值，在确定容差。参数的标定值仅影响次数损失而不影响出现数字位置。因此我们确定参数标定值时，不要考虑数字出现位置，要以次数损失达到最小为目标，采用式子 (1) ~ (3)；确定容差采用 *MonteCarlo* 模拟方法，由模型可知 t 值，次数损失采用精度表达式 (5)，然后对模型进行优化得到自己想要的。

(9) 采用 *MonteCarlo* 方法最终可得到成功概率 P 的近似值为 90.21%，在成功概率的前提下采用构建的 2048 游戏模型总共移动 1078 次完成游戏，达到 2048 值。

与模型一数据处理方式相同，通过 *SPSS* 数据处理可以确定，采用 *MonteCarlo* 方法得到的完成游戏成功率均值与移动次数均值与 *Java* 运行得到的成功率与次数误差较小，所以运行结果有效，移动次数与成功概率有说服力。

5.2 问题二的建立与求解

最大解情况分析要从概率方面考虑，分两种情况考虑。第一种情况是在排满方格前最后一个数字恰好是 2，第二种情况是在排满方格前最后一个数字恰好是 4。

5.2.1 最大解分析——最后出数为 2

在排满方格前最后一个数字恰好是 2 的情况下的最大解分析：
 分析，把它从单纯的 4×4 ，扩展到 $N \times N$ 的情况：
 1×1 的情况下，最大解为 4，如图。

4

2×2 的情况下，最大解为 $32 = 2^5 = 2^{(2 \times 2 + 1)}$ ，如图。

16	2
8	4

3×3 的情况下，最大解为 $1024 = 2^{10} = 2^{(3 \times 3 + 1)}$ ，如图。

512	16	8
256	32	4
128	64	2

4×4 的情况下，最大解为 $2^{16} = 2^{(4 \times 4)}$ ，如图。其中，我们知道最小的格必须是 4，所以有 $N - 14 = 2$ ，因此 $N = 16$ 。在这种情况下是能够全合并的，因此最高分可以得到 2^{16} 。

2^{16}	2^{12}	2^8	2^1
2^{15}	2^{11}	2^7	2^2
2^{14}	2^{10}	2^6	2^3
2^{13}	2^9	2^5	2^4

经过实际游戏可以得到最大值为 65536，与计算结果相符，如图 1 所示：

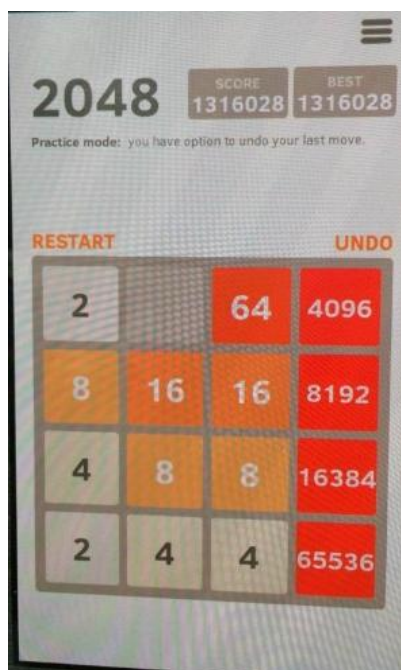


图 6 最后结果是 2 时达到的最大值

以此类推， $N * N$ 的情况下，最大解也就显然为 $2^{(N * N)}$ 。

5.2.2 最大解分析——最后出数为 4

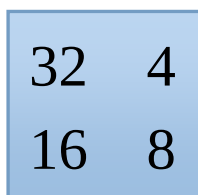
在排满方格前最后一个数字恰好是 4 的情况下的最大解分析：

分析，把它从单纯的 $4 * 4$ ，扩展到 $N * N$ 的情况：

$1 * 1$ 的情况下，最大解为 4，如图。



$2 * 2$ 的情况下，最大解为 $32 = 2^5 = 2^{(2 * 2 + 1)}$ ，如图。



$3 * 3$ 的情况下，最大解为 $1024 = 2^{10} = 2^{(3 * 3 + 1)}$ ，如图。



4*4 的情况下，最大解为 $2^{17} = 2^{(4*4 + 1)}$ ，如图。其中，我们知道最小的格必须是 4，所以有 $N - 14 = 2$ ，因此 $N = 16$ 。在这种情况下是能够全合并的，因此最高分可以得到 2^{17} 。

2^{17}	2^{10}	2^9	2^5
2^{16}	2^{11}	2^8	2^4
2^{15}	2^{12}	2^7	2^3
2^{14}	2^{13}	2^6	2^2

经过实际游戏可以得到最大值为 131072，与计算结果相符，如图 2 所示：

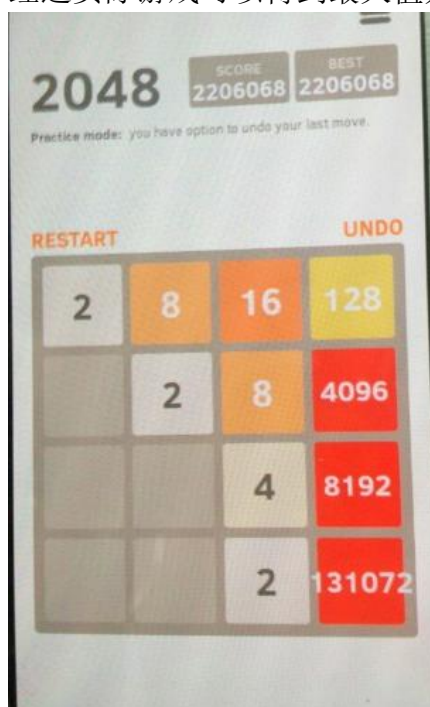


图 2 最后结果是 4 时达到的最大值

以此类推， $N * N$ 的情况下，最大解也就显然为 $2^{(N * N + 1)}$ 。

6. 模型的评价与推广

6.1 问题 1 模型的评价与推广

在问题 1 的处理过程中，对于求游戏 2048 通关的通用模型，就可以联系到双人对弈模型，把计算机当成对方，这样使得模型更加形象化。模型 1 中的 *Mini max* 算法虽然是一种悲观主义算法也可以称为最大最小准则，但在

Mini max 中玩家完全掌握主动,如果对方每一步决策都是完美的,那么玩家只能达到预计的最小损失格局,如果对方每一步决策并不都是完美的,则玩家就可以达到比之前预计最小损失格局的更好的结局。蒙特卡罗法,求出影响因素的概率分布,按计算机给定的概率分布生成大量的随机数,把随机数代入模型,求出目标变量的概率分布。对一些问题进行的理想化的分析,这与实际可能存在一定的差距。当然正是因为排除太多的意外的情况,才使得问题一模型得到很好的解决。

该题的模型不仅合理的解决了在不确定的因素的条件下,也就是概率随机性的情况下的问题,从而使人们在生活实际中,遇到不确定的事件后,做出最符合意愿的一个抉择。

6.2 问题 2 模型的评价与推广

问题 2 模型,先从最简单的问题入手,找到每增加格数对最大数之间存在的关系,从而得到有规律的公式,直观的得到最大数和格数之间的关系,使得模型一目了然。游戏 2048 是靠一个个生成的 2 和 4 加起来的,这里为了简化讨论,假设每一次移动产生的方格都是 2,进行理想化后找到了模型的递推关系,具有一定的理想化,使得模型存在一定的差距,得到的数值也是理论值。由于客观原因存在系统误差,且数据来源也有一定的误差,在解决实际问题的时候,应该多搜集可靠的数据,这样才能更好的解决问题。

同时,该模型为游戏开发商和玩家提供了一些游戏潜在的价值,对于游戏开发商来说可以制定让玩家能够接受的游戏难度,这样才能深受广大用户的欢迎。而对于玩家来说,通过此模型,便可知玩这个游戏是有头的,从而使玩家更加愿意去探索这个游戏的最终的结果,也就实现了这个游戏存在的意义。

7. 参考文献

- [1] 2048 Game, <https://github.com/ov3y/2048-AI>, 2014 年 5 月 25 日
- [2] 2048-AI github, <https://github.com/ov3y/2048-AI>, 2014 年 5 月 25 日
- [3] An Exhaustive Explanation of Minimax, a Staple AI Algorithm , <http://www.flyingmachinestudios.com/programming/minimax/>, 2014 年 5 月 25 日
- [4] Tic Tac Toe: Understanding the Minimax Algorithm , <http://www.neverstopbuilding.com/minimax>, 2014 年 5 月 25 日
- [5] CS 161 Recitation Notes - Minimax with Alpha Beta Pruning , <http://cs.ucla.edu/~rosen/161/notes/alphabeta.html>, 2014 年 5 月 25 日
- [6] 2048-AI 程序算法分析, <http://blog.codinglabs.org/articles/2048-ai-analysis.html>, 2014 年 5 月 25 日
- [7] What is the optimal algorithm for the game2048, <http://stackoverflow.com/questions/22342854/what-is-the-optimal-algorithm-for-the-game-2048>, 2014 年 5 月 25 日
- [8] 2048 在理论上是否可以无限玩下, <http://m.zhihu.com/question/23492860>, 2014 年 5 月 25 日

[9] 陈光亭 裘哲勇, 数学建模, 北京, 高等教育出版社, 2010 年 2 月。

8. 附录

代码 1—启发指标加权代码

```
// static evaluation function
AI.prototype.eval = function() {
    var emptyCells = this.grid.availableCells().length;

    var smoothWeight = 0.1,
        //monoWeight    = 0.0,
        //islandWeight = 0.0,
        mono2Weight    = 1.0,
        emptyWeight    = 2.7,
        maxWeight      = 1.0;

    return this.grid.smoothness() * smoothWeight
        //+ this.grid.monotonicity() * monoWeight
        //- this.grid.islands() * islandWeight
        + this.grid.monotonicity2() * mono2Weight
        + Math.log(emptyCells) * emptyWeight
        + this.grid.maxValue() * maxWeight;
};
```

代码 2—相关剪枝代码

```
// try a 2 and 4 in each cell and measure how annoying it is
// with metrics from eval
var candidates = [];
var cells = this.grid.availableCells();
var scores = { 2: [], 4: [] };
for (var value in scores) {
    for (var i in cells) {
        scores[value].push(null);
        var cell = cells[i];
        var tile = new Tile(cell, parseInt(value, 10));
        this.grid.insertTile(tile);
        scores[value][i] = -this.grid.smoothness() + this.grid.islands();
        this.grid.removeTile(cell);
    }
}

// now just pick out the most annoying moves
var    maxScore    =    Math.max(Math.max.apply(null,    scores[2]),
Math.max.apply(null, scores[4]));
```

```

for (var value in scores) { // 2 and 4
    for (var i=0; i<scores[value].length; i++) {
        if (scores[value][i] == maxScore) {
            candidates.push( { position: cells[i], value: parseInt(value, 10) } );
        }
    }
}

```

代码 3—设定时间搜索深度代码

```

// performs iterative deepening over the Alpha-Beta search
AI.prototype.iterativeDeep = function() {
    var start = (new Date()).getTime();
    var depth = 0;
    var best;
    do {
        var newBest = this.search(depth, -10000, 10000, 0 ,0);
        if (newBest.move == -1) {
            //console.log('BREAKING EARLY');
            break;
        } else {
            best = newBest;
        }
        depth++;
    } while ( (new Date()).getTime() - start < minSearchTime);
    //console.log('depth', --depth);
    //console.log(this.translate(best.move));
    //console.log(best);
    return best
}

```

代码 4——2048 C++实现代码

```

/*By Reason*/
#include<iostream>
#include <iomanip>
#include<math.h>
#include<stdlib.h>
#include<time.h>
#include <conio.h> //为了读取方向键
#include<windows.h>
using namespace std;
//srand( (unsigned)time( NULL ) ); //随机数种子 不能用在主函数
int pane[4][4]; //棋盘
int N=1; //2 的 n 次方
void showpane()//显示棋盘
{
    cout<<setw(46)<<"X2048 by Reason"<<endl;
}

```

```

        cout<<setw(50)<<" |-----|"<<endl;
        for(int i=0;i<=3;i++)
        {
            cout<<setw(24)<<"";
            for(int j=0;j<=3;j++)
            {
                //SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),FOREGR
                OUND_RED);
                if(pane[i][j]==0)
                    cout<<setw(2)<<"|"<<setw(4)<<" ";
                else
                    cout<<setw(2)<<"|"<<setw(4)<<pane[i][j];
                if(j==3)
                {
                    cout<<setw(2)<<"|"<<endl;
                    cout<<setw(50)<<" |-----|"<<endl;
                }
            }
        }
    }
}

void newgame() //开始游戏
{
    N=1;
    for(int i=0;i<=3;i++) //初始化棋盘
        for(int j=0;j<=3;j++)
            pane[i][j]=0;
    srand( (unsigned)time( NULL ) );
    int m=rand()%4;
    int n=rand()%4;
    int p=rand()%4;
    int q=rand()%4;
    pane[m][n]=pane[p][q]=2;
    showpane();
}

int if2n(int x) //判断 x 是否是 2 的 n 次方
{
    int flag=0;
    for(int n=1;n<=11;n++)
    {
        if(x==pow(2,n))
        {
            flag=1;
            if(n>N)
                N=n;
        }
    }
}

```

```

        return flag;
    }
}
return flag;
}
int upmove()    //上移
{
    int flag=0;
    for(int j=0;j<=3;j++)
        for(int i=0;i<3;i++)
        {
            if(if2n(pane[i][j]+pane[i+1][j])==1)
            {
                pane[i][j]=pane[i][j]+pane[i+1][j];
                pane[i+1][j]=0;
                flag=1;
            }
        }
    return flag;
}

int downmove() //下移
{
    int flag=0;
    for(int j=0;j<=3;j++)
        for(int i=3;i>0;i--)
        {
            if(if2n(pane[i][j]+pane[i-1][j])==1)
            {
                pane[i][j]=pane[i][j]+pane[i-1][j];
                pane[i-1][j]=0;
                flag=1;
            }
        }
    return flag;
}

int leftmove()  //左移
{
    int flag=0;
    for(int i=0;i<=3;i++)
        for(int j=0;j<3;j++)
        {
            if(if2n(pane[i][j]+pane[i][j+1])==1)
            {

```

```

        pane[i][j]=pane[i][j]+pane[i][j+1];
        pane[i][j+1]=0;
        flag=1;
    }
}
return flag;
}
int rightmove() //右移
{
    int flag=0;
    for(int i=0;i<=3;i++)
        for(int j=3;j>0;j--)
        {
            if(if2n(pane[i][j]+pane[i][j-1])==1)
            {
                pane[i][j]=pane[i][j]+pane[i][j-1];
                pane[i][j-1]=0;
                flag=1;
            }
        }
    return flag;
}
int testup() //能否上移测试
{
    int flag=0;
    for(int j=0;j<=3;j++)
        for(int i=0;i<3;i++)
        {
            if((if2n(pane[i][j]+pane[i+1][j])==1)&&pane[i+1][j])
            {
                flag=1;
            }
        }
    return flag;
}
int testdown() //测试能否下移
{
    int flag=0;
    for(int j=0;j<=3;j++)
        for(int i=3;i>0;i--)
        {
            if((if2n(pane[i][j]+pane[i-1][j])==1)&&pane[i-1][j])
            {
                flag=1;
            }
        }
    return flag;
}

```

```

        }
    }
    return flag;
}
int testleft() //测试能否左移
{
    int flag=0;
    for(int i=0;i<=3;i++)
        for(int j=0;j<3;j++)
        {
            if((if2n(pane[i][j]+pane[i][j+1])==1)&&pane[i][j+1])
            {
                flag=1;
            }
        }
    return flag;
}
int testright() //测试能否右移
{
    int flag=0;
    for(int i=0;i<=3;i++)
        for(int j=3;j>0;j--)
        {
            if((if2n(pane[i][j]+pane[i][j-1])==1)&&pane[i][j-1])
            {
                flag=1;
            }
        }
    return flag;
}
int panemax() //棋盘最大数
{
    int max=pane[0][0];
    for(int i=0;i<=3;i++)
        for(int j=0;j<=3;j++)
            if(pane[i][j]>max)
                max=pane[i][j];
    return max;
}
int ifwin() //判断是否胜利
{
    int flag=0;
    if(panemax()==2048)
    {

```

```

        cout<<setw(45)<<"You Win!"<<endl;
        flag=1;
    }
    return flag;
}
int ifGameOver()          //判断是否游戏结束
{
    int flag=0;
    if(testup()+ testdown() + testleft() + testright() == 0)
    {
        cout<<setw(43)<<"Game Over!"<<endl;
        flag=1;
    }
    return flag;
}
void addnewnumberup() //上移后添加新数
{
    srand( (unsigned)time( NULL ) );
    int n;
    if(N==1)
        n=1;
    else
        n=(rand()%(N)+1);
    int newnumber=pow(2,n);
    for(int i=3;i>=0;i--)
        for(int j=0;j<=3;j++)
            if(pane[i][j]==0)
            {
                pane[i][j]=newnumber;
                return;
            }
}
void addnewnumberdown() //下移后添加新数
{
    srand( (unsigned)time( NULL ) );
    int n;
    if(N==1)
        n=1;
    else
        n=(rand()%(N)+1);
    int newnumber=pow(2,n);
    for(int i=0;i<=3;i++)
        for(int j=0;j<=3;j++)
            if(pane[i][j]==0)

```

```

        {
            pane[i][j]=newnumber;
            return;
        }
    }
void addnewnumberleft()//左移后添加新数
{
    srand( (unsigned)time( NULL ) );
    int n;
    if(N==1)
        n=1;
    else
        n=(rand()%(N)+1);
    int newnumber=pow(2,n);
    for(int j=3;j>=0;j--)
        for(int i=0;i<=3;i++)
            if(pane[i][j]==0)
            {
                pane[i][j]=newnumber;
                return;
            }
}
void addnewnumberright() //右移后添加新数
{
    srand( (unsigned)time( NULL ) );
    int n;
    if(N==1)
        n=1;
    else
        n=(rand()%(N)+1);
    int newnumber=pow(2,n);
    for(int j=0;j<=3;j++)
        for(int i=0;i<=3;i++)
            if(pane[i][j]==0)
            {
                pane[i][j]=newnumber;
                return;
            }
}
int GetDirection() //读取方向
{
    int ret = 0;
    do
    {

```

```

        int ch = _getch();
        if(isascii(ch))
            continue;
        ch = _getch();
        switch(ch)
        {
        case 72:
            ret = 2; // top
            break;
        case 75:
            ret = 1; // left
            break;
        case 77:
            ret = 3; // right
            break;
        case 80:
            ret = 4; // down
            break;
        default:
            break;
        }
    } while (ret == 0);

    return ret;
}
int main() //主函数
{
    system("color f9");
    int makesure=1;
    while(makesure)
    {
        system("cls");
        newgame();
        while(ifwin()+ifGameOver()==0)
        {
            int c=GetDirection();
            switch(c)
            {
            case 2:
                if(testup()==1)
                {
                    upmove();
                    addnewnumberup();
                    system("cls");
                    showpane();
                }
            }
        }
    }
}

```

```

        }
        break;
    case 4:
        if(testdown()==1)
        {
            downmove();
            addnewnumberdown();
            system("cls");
            showpane();
        }
        break;
    case 1:
        if(testleft()==1)
        {
            leftmove();
            addnewnumberleft();
            system("cls");
            showpane();
        }
        break;
    case 3:
        if(testright()==1)
        {
            rightmove();
            addnewnumberright();
            system("cls");
            showpane();
        }
        break;
    default:
        break;
    }
}
cout<<setw(43)<<"你的最后成绩为: "<<panemax()<<endl;
cout<<setw(60)<<"若要重新开始游戏请输入 1，若要结束请输入 0。
"<<endl;
cin>>makesure;
while(makesure!=1&&makesure!=0)
{
    cout<<"输入不正确，请重新输入!"<<endl;
    cin>>makesure;
}
}
cout<<"再见！"<<endl;

```

```
system("pause");  
return 0;  
}
```

实验验证得到的图片：

